

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250279879

Kind Code

A1

Publication Date

September 04, 2025

Inventor(s)

Pishdadian; Hamid et al.

Proof of Entanglement and Cipher that Carries its own Key

Abstract

What is being presented is Proof of Entanglement (PoE), an algorithmic interchange of information that removes the need for classical consensus when adding blocks to a blockchain through its inherent ability to entangle the source of a document (sender) to the consumer of the document (receiver). The presented method and algorithm will ensure that only the intended receiver can possibly receive and be able to decrypt the document. Furthermore, PoE will guarantee the authenticity and security of the document to a level of entropy that will take more computational energy than is available on Earth to decipher, using today's supercomputers. PoE will create a quantum link (shared encryption key) between the sender and receiver. This key is used for document encryption and decryption but is never sent from the sender to the receiver. It is generated by the platform's servers and link devices (LINK) for both the representative of the sender and the representative of the receiver. This encryption key is produced by data held in the LINK that provides secure data to their counterpart entangled node devices (NODE). These in turn create the actual block of the blockchain. FIG. 1 is the block diagram of the platform. It consists of the following servers, LINK, and NODE. The NODE and LINK are held by people and/or businesses that support the platform. The platform description will clarify some of the complexities associated with the model.

Inventors: Pishdadian; Hamid (Warwick, RI), Khalis; Akram (Roselle, NJ)

Applicant: Pishdadian; Hamid (Warwick, RI); Khalis; Akram (Roselle, NJ)

Family ID: 96880640

Appl. No.: 18/885651

Filed: September 14, 2024

Related U.S. Application Data

us-provisional-application US 63526842 20230714

Publication Classification

Int. Cl.: H04L9/08 (20060101); G06K19/06 (20060101); H04L9/32 (20060101)

Background/Summary

BACKGROUND OF THE INVENTION

[0001] Blockchains have been in existence for many years; but their applications have flourished since the rise of Bitcoin and Ethereum networks, as well as other discoveries and innovations. Today, blockchains are used for storing secure transactions, holding user information, over-the-border currency transfer, distributed applications, financial technologies, and many other services. Proof of Work (PoW) is the mechanism by which many of these blockchains verify the authenticity of the transactions in order to come to a consensus among chain holders. The problem with PoW is the energy required to secure the transactions. A typical PoW consensus will consume as much as 1449 KWH of power. That is roughly the same power to energize 300 homes in the USA. Proof of Stake (POS) is the second most popular consensus mechanism. PoS uses much less power than PoW, but it is still substantial. There are other methods of consensus that perform better than PoW and PoS, but there are structural problems with all of them. To make blockchain technology what it promises to be requires a method of consensus that is fast, reliable, uses minimal energy, and—above all—is very secure. In the abstract portion of this application, we will introduce a mechanism that creates a communication scheme that achieves all of these requirements.

Description

PLATFORM DESCRIPTION

[0002] The underlying platform that runs PoE consists of the following servers and characteristics:

[0003] NODE: These hardware devices hold the blockchain.

[0004] LINK: The link between cloud servers and NODE. LINK and NODE use Simulated Quantum Entanglement to communicate. US patent publication number: 20220385472

[0005] Client servers: Entry points for all mobile devices connected to these servers.

[0006] API servers: Entry point for all non-mobile devices into the platform.

[0007] Link servers: These servers are the hubs for all LINKs and NODEs. LINK servers do the computation to be used in verification and verify user IDs, known as SQid. They also are the bridge that links the representatives of senders to the representative of the receivers.

[0008] Cloud database servers: Database servers hold the encrypted user data.

[0009] Dispatch servers: Hubs for all servers to communicate using PoE for security.

[0010] Audit servers: These servers communicate with NODEs and database servers to get information about all activities on the platform. They also publish the platform's main blockchain.

[0011] Virgin ID (Vid): As shown in FIG. 2, there are a total of 79 types of LINKs and a number of NODEs that are associated with each LINK. There can be as many as 256 batches of these NODEs. Every Vid contains a set of (79 CHOOSE 72) possible LINK identifiers. A set of these ids is called Virgin ID. LINK IDs range from 1 to 79, therefore 7 bits are needed to represent each LINK ID. Therefore, the Vid is made of $7 \times 72 + 8$ or 512 bits. The value 8 represents the batch number that the Vid belongs to. Vid is used to specify an entity on the platform.

[0012] User ID (SQid): The heart of PoE is the identification code assignment to users who wish to use the platform. As shown in Table 1, SQid consists of various elements. It carries its own key that is used to verify its authenticity. It is a 1024-bit number with 256-bit as the key. This key is generated by parameters that are kept securely in the LINK devices. The Vid along with other private information associated with any given user is combined and hashed as part of the SQid. During the deciphering process, the key is used to decrypt the information kept in the cloud database that is assigned to the user associated with an SQid. Only the LINK devices hold the information required to produce the key and decrypt the data,

which holds the actual Vid of the user. The following mathematical relationships denote this process.

LINK devices contain several parameters: [0013] P: a randomly assigned 32-bit prime number:

$P < (2^{\text{sup.}32} - 1)$ [0014] A: a randomly assigned 32-bit integer. [0015] B: a randomly assigned 32-bit

integer. [0016] C: a randomly assigned 32-bit integer. [0017] D: a randomly assigned 32-bit integer.

[0018] ID = {1, 2, 3, . . . 79} Set of first 79 integers. [0019] Vid is a unique subset of LINK IDs {72, 79}

[0020] Nid is a unique subset of LINK IDs {7, 79} [0021] Vid intersect Nid = { } [0022] The intersection

of Vid and Nid is an empty set. [0023] Let's name the members of Nid set L1 to L7 [0024] Nid = {L1, L2,

L3, L4, L5, L6, L7}

[0025] Each member of the Nid set includes the following parameters: [0026] P, A, B, C, D. Specifically, P1, A1, B1, C1, D1 to P7, A7, B7, C7, D7.

[0027] C and D parameters are randomly selected aliases of P and B parameters:

[0028] The database encryption key for a given Vid is calculated as follows:

[00001] $K_0 = 1$ $K_1 = (P_1 \times K_0 + A_1) \text{ xor } B_1$ $K_2 = (P_2 \times K_1 + A_2) \text{ xor } B_2$

Continue for all 7 LINKs.

[00002] $K_7 = (P_7 \times K_6 + A_7) \text{ xor } B_7$ IDKEY = K_7

[0029] IDKEY is the actual key that is included as part of SQid. The maximum size of the IDKEY is $2^{\text{sup.}32} \times 7$ or 224.

[0030] The IDKEY is padded to 256 bits.

Encryption Key Generation to Encrypt the Document:

[0031] The Encryption Key is calculated based on the C and D parameters as follows:

[00003] $E_0 = 1$ $E_1 = C_0 \times E_0 + D_1$ $E_2 = C_1 \times E_1 + D_2$

Continue for all 7 LINKs.

[00004] $E_7 = C_7 \times E_6 + D_7$ Kenc = E_7

[0032] Since the actual values of C and D are never published or transmitted and have no mathematical relationship to counterparts P and A, there is no way to discover Kenc due to IDKEY. However, with an IDKEY (Key), one can find whether a given LINK is one of the contributors to the Key value. This can only happen if the values of P, A, and B are known for all LINK or if they have access to the servers that communicate with the LINK. These are only available in a distributed network, and only client servers can communicate with LINK servers.

The Key Verification Process:

[0033] Iteratively the value $K_{\text{sub}.n}$ and $E_{\text{sub}.n}$ are provided to the ID set from last to the first. [0034]

$K_{\text{sub}.n} = \text{IDKEY}$. [0035] $E_{\text{sub}.n} = 1$.

[0036] LINK in turn evaluates the value K_{n-1} . If the remainder R is zero, then it returns the given value K_{n-1} and E_{n-1} . Otherwise, it returns the new value $K_{\text{sub}.n}$ and $E_{\text{sub}.n}$,

[00005] $K_{n-1} = (K_n \text{ xor } C_n) / P_n + R$ $E_{n-1} = E_n \times C_n + D_n$

[0037] When all seven members of Nid return their evaluation, the final value $E_{\text{sub}.final}$ is the key that decrypts the cloud data of a given SQid.

[0038] The user data kept in the cloud are then hashed and checked against the hashed portion of SQid. A match will verify the authenticity of the key.

Proof of Entanglement (PoE):

[0039] The PoE process is much like the ID verification process. However, what is unique about PoE is the relationship that exists between any two given SQids. This relationship is used to create an entanglement key whereby the sender can encrypt the document, and it can only be decrypted by the receiver. The SQid verification process guarantees that only the true SQid can access the specific data stored in LINK devices in order to reproduce the exact entanglement key. Furthermore, the following relationship calculates the entropy of the shared key. Due to the random nature of parameters P, A, B, C, and D, Shor's algorithm-used by quantum computers-cannot be used in attempted decryption. For this reason, PoE is a quantum-proof algorithm. [0040] ID = {1, 2, 3, . . . 79} [0041] Set of first 79 integers.

[0042] Vid1 unique subset of {72, 79} [0043] A unique subset of LINK IDs. Belonging to SQid1 [0044]

Vid2 unique subset of {72, 79} [0045] A unique subset of LINK IDs. Belonging to SQid2 [0046] Uid

intersect {Vid1, Vid2} The intersection of sets Vid1 and Vid2.

[00006] Eid = ID - Uid

[0047] The set that does not include Uid.

[0048] Depending on Vid1 and Vid2, Eid has 7 to 14 members. The client server representing SQid1 has access to its own SQid and the SQid of the receiver. With these two user IDs, the client-server can receive Vid1 and Vid2 via the ID verification process, through link servers. It then uses the Eid set and the following relationships to calculate the encryption key that links these two SQids.

KeyCalculation

$$K_0 = 1$$

KeySizeCalculation

$$e_0 = 0$$

$$[00007] K_1 = (P_1 \times K_0 + A_1) \text{xor} B_1$$

$$e_1 = (2^{**} 16) * (2^{**} 16) = (2^{**} 32)$$

$$K_2 = (P_2 \times K_1 + A_2) \text{xor} B_2$$

$$e_2 = e_1 + (2^{**} 16) * (2^{**} 16) = (2^{**} 32) + (2^{**} 17)$$

Continue for all Members of Eid.

$$[00008] K_n = (P_n \times K_{n-1} + A_n) \text{xor} B_n \quad e_n = e_{n-1} + 2^{(n+1)*17} \quad [0049] \text{ for } n > 1$$

[0050] The subscript n has a min value of 7 and a max value of 14. The key size is the average of individual key sizes:

$$\text{TABLE-US-00001 } e_{\text{sub.7}} = 32 + 6 * 17 = 134 \quad e_{\text{sub.8}} = 32 + 7 * 17 = 151 \quad e_{\text{sub.9}} = 32 + 8 * 17 = 168$$

$$e_{\text{sub.10}} = 32 + 9 * 17 = 185 \quad e_{\text{sub.11}} = 32 + 10 * 17 = 202 \quad e_{\text{sub.12}} = 32 + 11 * 17 = 219 \quad e_{\text{sub.13}} = 32 + 12 * 17 = 236 \quad e_{\text{sub.14}} = 32 + 13 * 17 = 253$$

[00009]

$$E = (e_7 + e_8 + e_9 + e_{10} + e_{11} + e_{12} + e_{13} + e_{14}) / 7 \quad E = (134 + 151 + 168 + 185 + 202 + 219 + 236 + 253) / 7$$

[0051] Therefore, the size of the PoE encryption key is 185 bits. It will be strategically padded to 256 bits. Any symmetric encryption algorithm such as AES-256 may be used to encrypt and decrypt the document being sent from the sender to the receiver.

Sample Transaction:

Claims

1. What is being claimed as shown in FIG. 2, Table 1, and described in sections **10**, **11**, and **12** in the platform description, is that it is possible to create a public user ID that can carry its own verification, and the verification key can be used to securely decrypt the content of the user information.
2. Using PoE as described in section **13**, a transaction can be encrypted in one server and sent to another remote server without a direct connection, so the PoE mechanism indirectly provides the encryption key to both parties.
3. In claim 1, the public ID (SQid) contains cryptographic information about its owner.
4. In claim 1, the public ID (SQid) may or may not contain secure information about the location and address of data in databases and/or blockchains.
5. In claim 1, the public ID (SQid) may be independently secured as a QR code and scanned by third-party software.
6. In claim 5, the ability to independently secure the public ID (SQid) as a QR code and scan it with third-party software ensures convenient and reliable verification processes.
7. In claim 1, the public ID (SQid) can be used as a secure authentication mechanism for various physical and digital assets.
8. In claim 1, SQid may be used to identify inanimate objects.
9. In claim 1, SQid may contain information that describes the type of data or object being referred to.
10. In claim 2, Proof of Entanglement can be used as a bilateral secure link between two endpoints, and this cryptographic link removes the need for Diffie-Hellman or any other type of key exchange.
11. In claim 2, Proof of Entanglement can be used as a bilateral link between people and inanimate objects.
12. In claim 2, the shared key is not detectable or calculated by any one party, and it is generated by the platform at the time of usage and destroyed after usage is completed.
13. In claim 2, the cryptographic shared key is not calculatable by shore's algorithm since it is not cyclic by design.
14. In claim 2, the shared key generated by PoE may be recorded in a database or a blockchain with the

approval of both parties only.

15. In claim 2, the generated shared key is secured against all present methods of cryptographical discoveries due to the sheer size of the key in term of its entropy, and the complexity of the generated key is 2 to power 234 bits or $10e70$.
